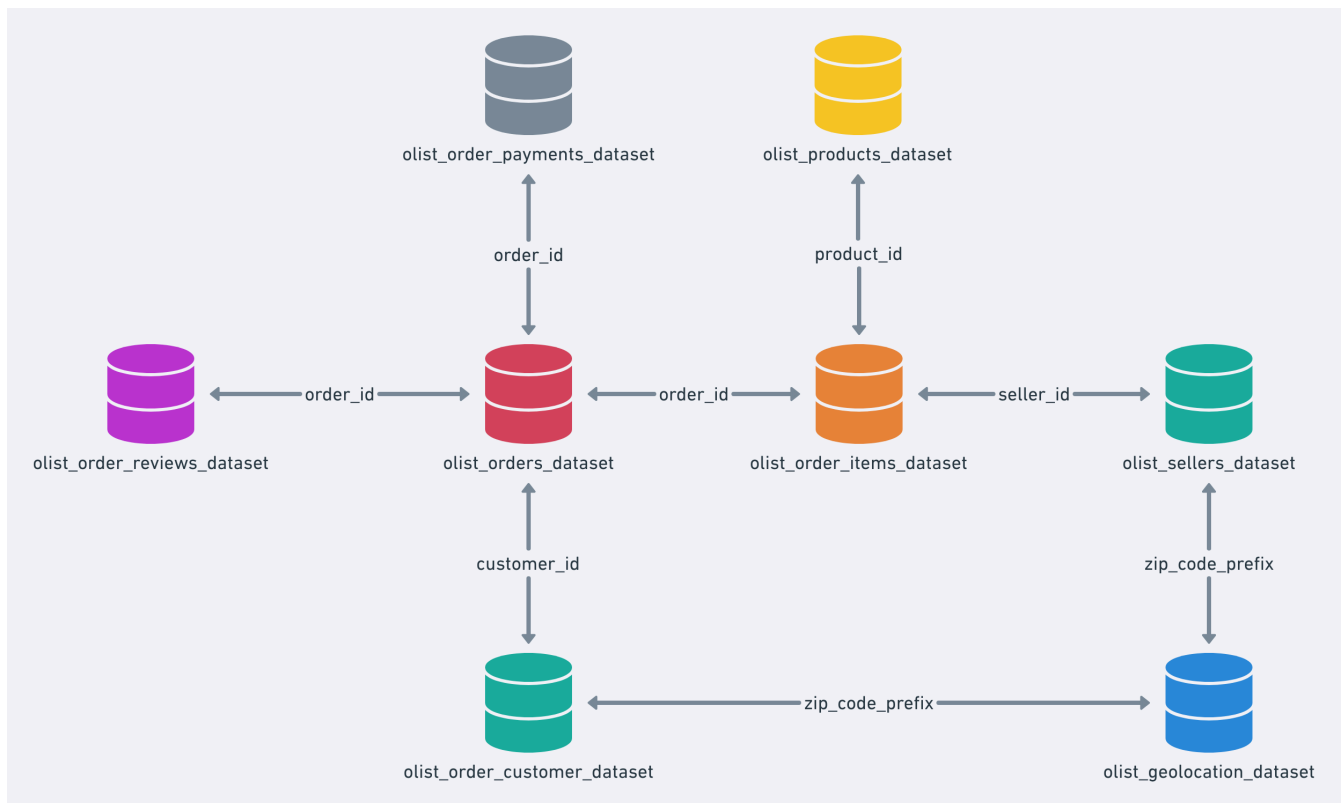


----- Brazilian E-Commerce Public Dataset SQL Analysis -----



Module & Data Loading

Module Loading

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import os
import sqlite3
from datetime import datetime
import warnings
warnings.filterwarnings("ignore")
```

Data Loading

```
import kagglehub
path = kagglehub.dataset_download("olistbr/brazilian-ecommerce")
```

Using Colab cache for faster access to the 'brazilian-ecommerce' dataset.

```
df_orders = pd.read_csv(path+'/olist_orders_dataset.csv')
df_items = pd.read_csv(path+'/olist_order_items_dataset.csv')
df_products = pd.read_csv(path+'/olist_products_dataset.csv')
```

local SQL database connection

```
conn = sqlite3.connect('olist.db')
```

✓ Dump the data into the SQL database

```
df_orders.to_sql('orders', conn, if_exists='replace', index=False)
df_items.to_sql('items', conn, if_exists='replace', index=False)
df_products.to_sql('products', conn, if_exists='replace', index=False)
```

```
32951
```

✓ Test Test Test

```
query = "SELECT * FROM orders LIMIT 1"
print(pd.read_sql(query, conn))
```

```
      order_id      customer_id \
0  e481f51cbdc54678b7cc49136f2d6af7  9ef432eb6251297304e76186b10a928d

      order_status order_purchase_timestamp  order_approved_at \
0      delivered      2017-10-02 10:56:33  2017-10-02 11:07:15

      order_delivered_carrier_date order_delivered_customer_date \
0      2017-10-04 19:55:00      2017-10-10 21:25:13

      order_estimated_delivery_date
0      2017-10-18 00:00:00
```

```
query = "SELECT * FROM products LIMIT 1"
print(pd.read_sql(query, conn))
```

```
      product_id product_category_name \
0  1e9e8ef04dbcff4541ed26657ea517e5      perfumaria

      product_name_lenght product_description_lenght product_photos_qty \
0              40.0              287.0              1.0

      product_weight_g product_length_cm product_height_cm product_width_cm
0              225.0              16.0              10.0              14.0
```

```
query = "SELECT * FROM items LIMIT 1"
print(pd.read_sql(query, conn))
```

```
      order_id order_item_id \
0  00010242fe8c5a6d1ba2dd792cb16214      1

      product_id      seller_id \
0  4244733e06e7ecb4970a6e2683c13e61  48436dade18ac8b2bce089ec2a041202

      shipping_limit_date price freight_value
0  2017-09-19 09:45:35      58.9      13.29
```

✓ EDA

✓ What are the Top 10 Product Categories by Total Revenue?

```
print('Top 10 Product Categories by Total Revenue')
print("-----")

# -----

top_10_query = """SELECT product_category_name as Category, SUM(price) as Revenue FROM products
LEFT JOIN items ON products.product_id = items.product_id
LEFT JOIN orders ON items.order_id = orders.order_id
WHERE order_status = 'delivered'
GROUP BY product_category_name
ORDER BY Revenue DESC
LIMIT 10"""
print(pd.read_sql(top_10_query, conn))
```

Top 10 Product Categories by Total Revenue

	Category	Revenue
0	beleza_saude	1233131.72
1	relogios_presentes	1166176.98
2	cama_mesa_banho	1023434.76
3	esporte_lazer	954852.55
4	informatica_acessorios	888724.61
5	moveis_decoracao	711927.69
6	utilidades_domesticas	615628.69
7	cool_stuff	610204.10
8	automotivo	578966.65
9	brinquedos	471286.48

✓ "Health & Beauty" (beleza_saude) Time Series

```
print('Health & Beauty (beleza_saude) Time Series')
print("-----")

# -----

Timeseries_query = """SELECT strftime('%Y-%m',order_purchase_timestamp) as Date, SUM(price) as Revenue FROM products
LEFT JOIN items ON products.product_id = items.product_id
LEFT JOIN orders ON items.order_id = orders.order_id
WHERE order_status = 'delivered' and product_category_name = 'beleza_saude'
GROUP BY Date
ORDER BY Date"""
print(pd.read_sql(Timeseries_query, conn))
```

Health & Beauty (beleza_saude) Time Series

	Date	Revenue
0	2016-09	134.97
1	2016-10	3439.25
2	2017-01	12366.44
3	2017-02	22059.92
4	2017-03	25652.45
5	2017-04	22156.34
6	2017-05	46340.06
7	2017-06	31676.81
8	2017-07	34239.45
9	2017-08	49030.23
10	2017-09	50649.16
11	2017-10	40698.99
12	2017-11	78274.40
13	2017-12	60688.75
14	2018-01	71405.60
15	2018-02	84576.55
16	2018-03	84488.57
17	2018-04	91058.45
18	2018-05	94534.38
19	2018-06	106745.72
20	2018-07	103524.22
21	2018-08	119391.01

Revenue went from 134 USD (Sept 2016) to 119,000 USD (Aug 2018). This wasn't a "one-time spike"; it is a dominating category.

Notice 2017-11 (\$78k). That is Black Friday

✓ which states are the slowest to deliver to

```
df_customers = pd.read_csv(path+'/olist_customers_dataset.csv')
df_customers.to_sql('customers', conn, if_exists='replace', index=False)
```

99441

```
query = "SELECT * FROM customers LIMIT 1"
print(pd.read_sql(query, conn))
```

	customer_id	customer_unique_id	customer_zip_code_prefix	customer_city	customer_state
0	06b8999e2fba1a1fbc88172c00ba8bc7	861eff4711a542e4b93843c6dd7febb0	14409	franca	SP

```

print('which states are the slowest to deliver to')
print("-----")

# -----

delivery_query = """SELECT AVG((julianday(order_delivered_customer_date) - julianday(order_purchase_timestamp))) as Delay,
LEFT JOIN customers ON orders.customer_id = customers.customer_id
WHERE order_status = 'delivered'
GROUP BY state
ORDER BY Delay DESC
LIMIT 5"""
print(pd.read_sql(delivery_query, conn))

```

```

which states are the slowest to deliver to
-----
      Delay state
0  29.387546   RR
1  27.185068   AP
2  26.425991   AM
3  24.543855   AL
4  23.772917   PA

```

top 3 states: RR (Roraima), AP (Amapá), AM (Amazonas) -> are all in the Northern Region (Amazon Basin)
Customers here wait nearly one month (29 days) for a package.

▼ RFM

Who are our Top 5 customers based on Total Spending?

```

print('Who are our Top 5 customers based on Total Spending?')
print("-----")

# -----

customer_query = """SELECT customer_unique_id as Customer, SUM(price) as Total_Spent FROM orders
LEFT JOIN items ON orders.order_id = items.order_id
LEFT JOIN customers ON orders.customer_id = customers.customer_id
WHERE order_status = 'delivered'
GROUP BY Customer
ORDER BY Total_Spent DESC
LIMIT 5"""
print(pd.read_sql(customer_query, conn))

```

```

Who are our Top 5 customers based on Total Spending?
-----
      Customer      Spent
0  0a0a92112bd4c708ca5fde585afaa872  13440.0
1  da122df9eeddfedc1dc1f5349a1a690c   7388.0
2  763c8b1c9c68a0229c42c9fc6f662b93   7160.0
3  dc4802a71eae9be1dd28f5d788ceb526   6735.0
4  459bef486812aa25204be022145caa62   6729.0

```

The customer *0a0a92112bd4c708ca5fde585afaa872* spent R\$ 13,440

